

รายงานโครงงานวิชา

วิธีวิทยาการทำโปรแกรม 1 (2110215)

Tower Defense Game

ชื่อกลุ่ม	EKae
สมาชิกกลุ่ม	นายธรา วรานุเศรษฐ์ รหัสนิสิต 6630132921
	นางสาวพิชญ์สินี เกื้อนถนอม รหัสนิสิต 6730371621
ภาคการศึกษา	ภาคการศึกษาปลาย ปีการศึกษา 2568

สารบัญ

สารบัญ

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

1.2 วัตถุประสงค์

1.3 เทคโนโลยีที่ใช้

บทที่ 2 UML

ไฟล์ที่ 1: Entity + Tower + Enemy hierarchy

ไฟล์ที่ 2: Model + Controller + Thread

ไฟล์ที่ 3: UI + Util

บทที่ 3 วิธีการใช้งานโปรแกรม

3.1 ความต้องการของระบบ (System Requirements)

3.2 การติดตั้งและรันโปรแกรม

3.2.1 วิธีรันจาก JAR file

3.3 หน้าจอหลักของโปรแกรม

3.4 HUD (Head-Up Display)

3.5 ประเภทของ Tower

3.6 วิธีวาง Tower

3.7 ประเภทของ Enemy

3.8 ระบบ Wave

3.9 เงื่อนไขการชนะและแพ้

การแพ้ (Game Over)

กลยุทธ์แนะนำ

3.10 การควบคุมเสียง

บทที่ 4 การออกแบบเชิงวัตถุ (OOP)

4.1 Encapsulation and Access Modifier

4.1.1 การใช้ Access Modifier

4.1.2 ตัวอย่างโค้ด

4.2 Inheritance

4.2.1 โครงสร้าง Inheritance

4.2.2 เหตุผลที่ใช้ Inheritance

4.2.3 ตัวอย่างโค้ด

4.3 Polymorphism

4.3.1 ตัวอย่าง Polymorphism ที่ใช้ในโครงงาน

4.3.2 ตัวอย่างโค้ด

4.4 Abstraction และ Interface

4.4.1 Interface ที่ใช้

4.4.2 เหตุผลที่ใช้ Interface

4.4.3 ตัวอย่างโค้ด

4.5 Thread

4.5.1 Thread ที่ใช้ในโครงงาน

4.5.2 เหตุผลที่ใช้ 2 Thread แยกกัน

4.5.3 ตัวอย่างโค้ด

บทที่ 5 รายละเอียด Class ในโครงงาน

5.1 ภาพรวม Package และ Class

5.2 Interfaces

Attackable

Damageable

Renderable

5.3 Model Classes

Entity (abstract)

Tower (abstract)

ArrowTower / CannonTower / IceTower

Enemy (abstract)

BasicEnemy / FastEnemy / BossEnemy

GameMap

Projectile และ SplashProjectile

5.4 Controllers

GameController

WaveController

5.5 Threads

GameLoopThread

SpawnThread

5.6 Utility Classes

SoundManager

SpriteSheet

บทที่ 6 การทดสอบโปรแกรม

6.1 Class ที่ทดสอบ

6.2 ผลการทดสอบ

บทที่ 7 ข้อจำกัดและแนวทางการพัฒนาต่อ

7.1 สรุปภาพรวม

7.2 รายละเอียดแต่ละหัวข้อ

7.2.1 ปุ่ม Reset เกม

7.2.2 ปุ่มถอน Tower

7.2.3 พฤติกรรม Projectile เมื่อ Target ตาย

7.2.4 Fast Forward

7.2.5 Game Balance

7.3 ข้อจำกัดที่รับรู้

ภาคผนวก

[ก. JavaDoc](#)

[ข. GitHub Repository](#)

[ค. แหล่งที่มาของ Asset](#)

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

Tower Defense Game 2D มองจากมุมบน (Top-down view) เป็นโครงงานที่พัฒนาขึ้นเพื่อศึกษาและประยุกต์ใช้หลักการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) ในภาษา Java ร่วมกับ JavaFX สำหรับสร้างส่วนติดต่อผู้ใช้แบบ GUI และ Thread สำหรับการประมวลผลแบบพร้อมกัน โดยแนวคิดของเกมคือผู้เล่นจะต้องวางป้อม (Tower) เพื่อป้องกันไม่ให้ศัตรู (Enemy) เดินผ่านเส้นทางมาถึงฐานได้

1.2 วัตถุประสงค์

- เพื่อฝึกการออกแบบและพัฒนาโปรแกรมโดยใช้หลักการ OOP ได้แก่ Encapsulation, Inheritance, Polymorphism และ Abstraction
- เพื่อศึกษาการพัฒนา GUI ด้วย JavaFX
- เพื่อประยุกต์ใช้ Thread ในการพัฒนาระบบ
- เพื่อฝึกการเขียน Unit Test ด้วย JUnit 5

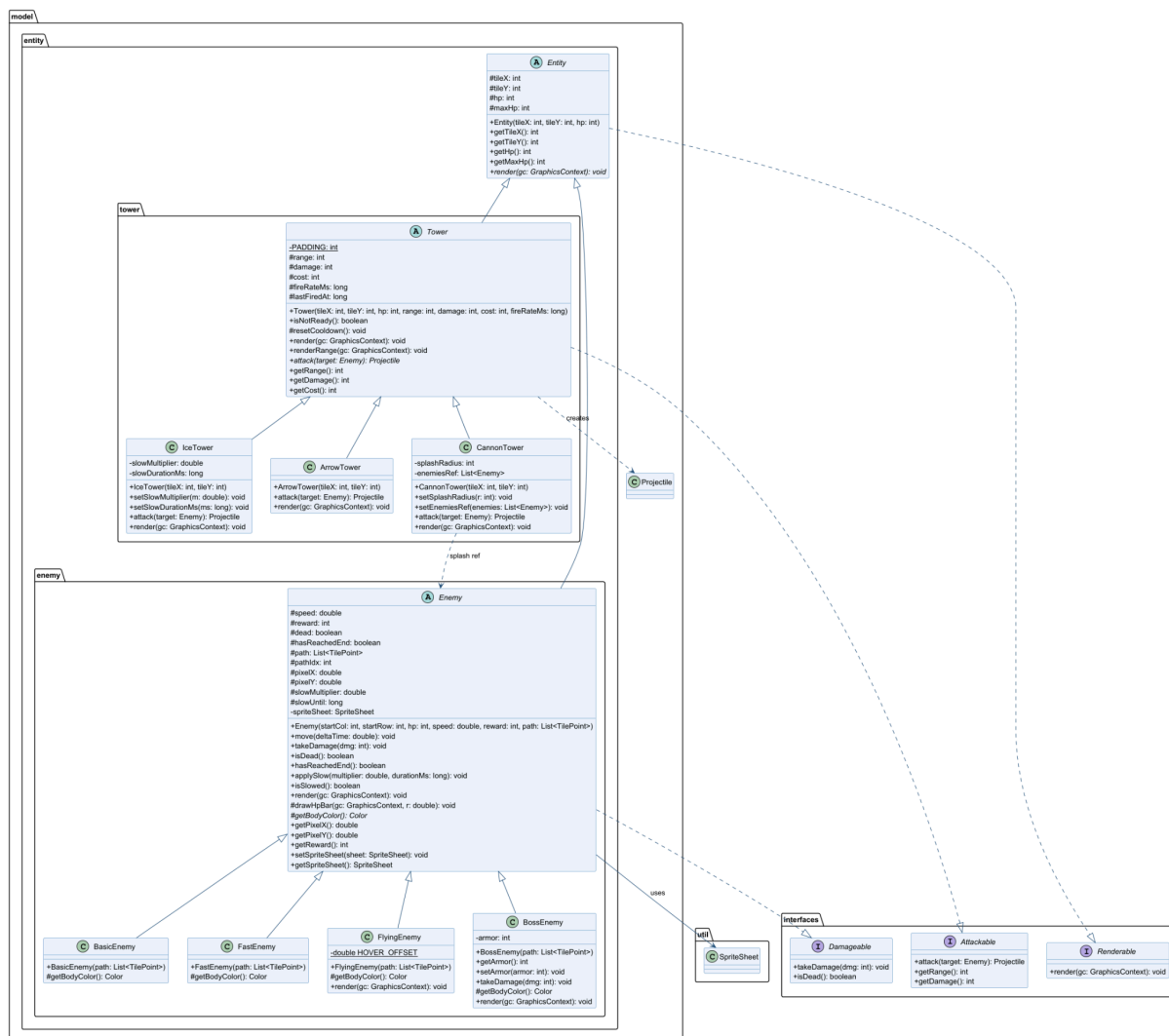
1.3 เทคโนโลยีที่ใช้

เทคโนโลยีที่ใช้	การนำไปใช้
JDK 24.0.1	ชุดเครื่องมือที่ใช้พัฒนาซอฟต์แวร์หลัก
JavaFX 24.0.1	สร้างส่วนติดต่อผู้ใช้แบบ GUI
JUnit 5	Unit Testing ครอบคลุม logic หลักทั้งหมด
IntelliJ IDEA	IDE ที่ใช้พัฒนาซอฟต์แวร์
GitHub	Version Control และส่งงาน

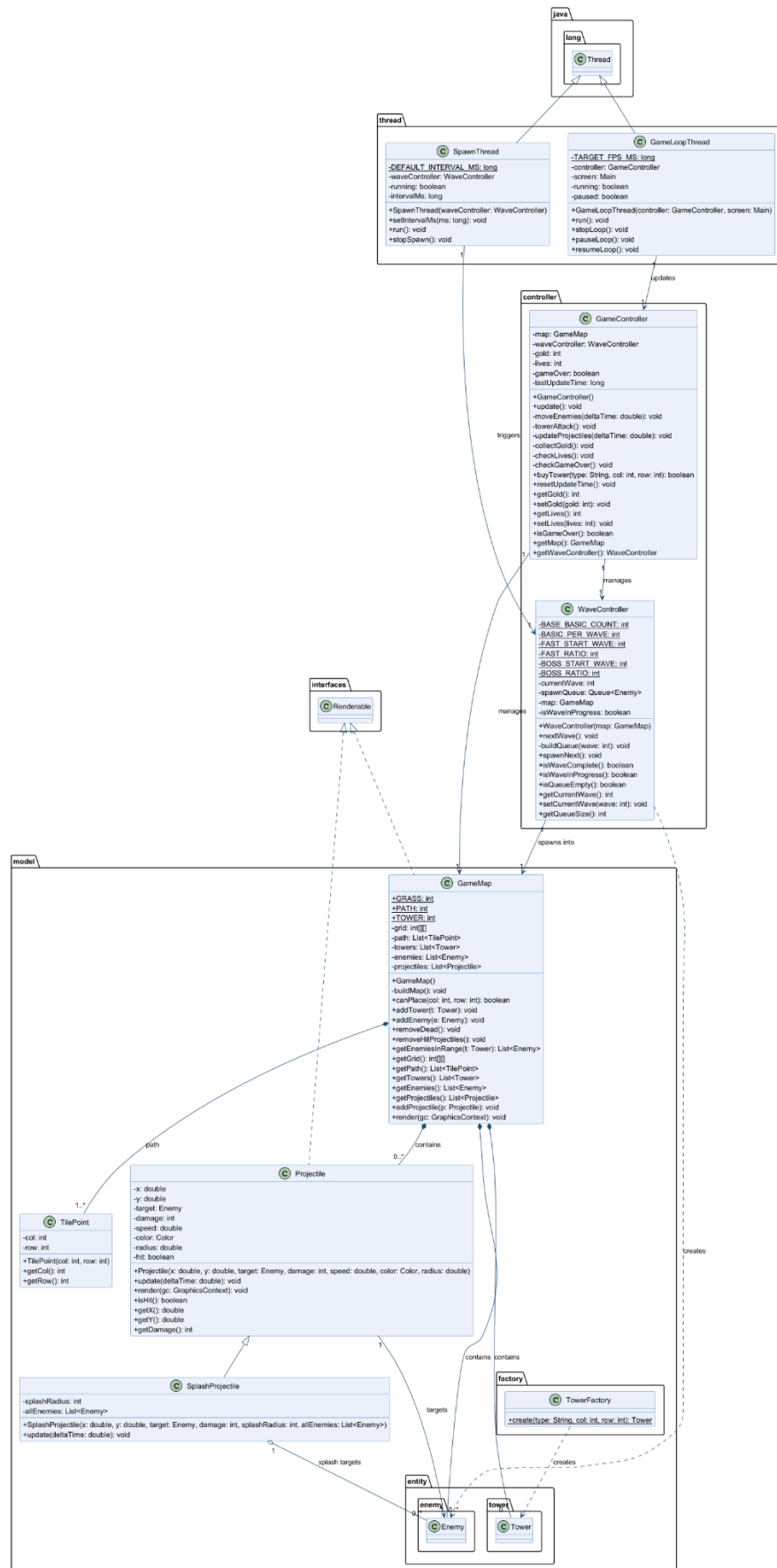
บทที่ 2 UML

ถูกแบ่งเป็น 3 ไฟล์ตาม package เพื่อลดความซับซ้อน

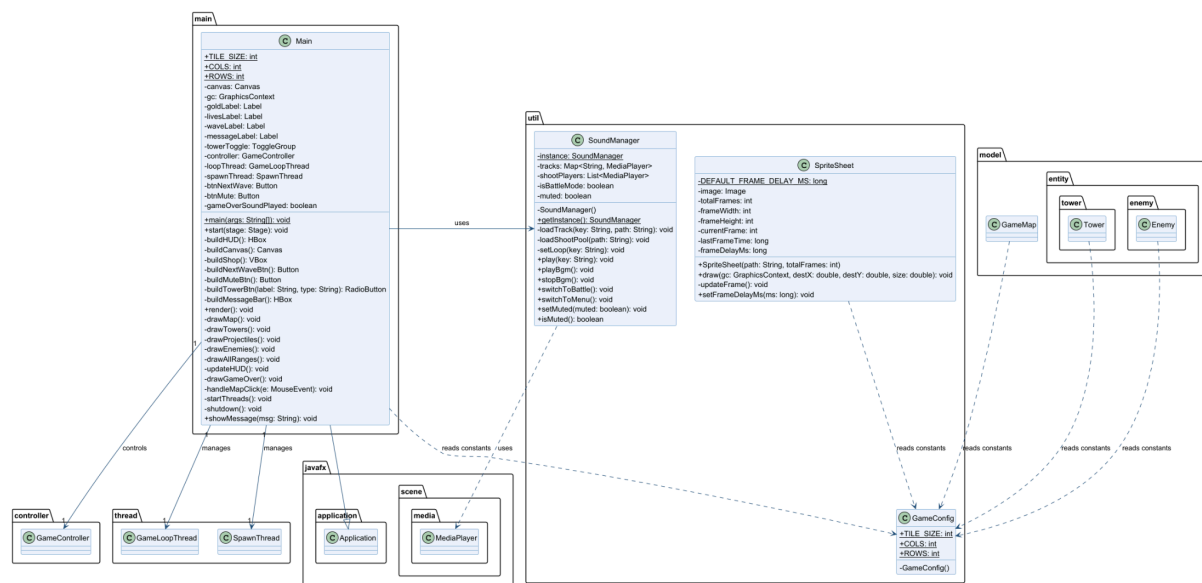
ไฟล์ที่ 1: Entity + Tower + Enemy hierarchy



ไฟล์ที่ 2: Model + Controller + Thread



ไฟล์ที่ 3: UI + Util



บทที่ 3 วิธีการใช้งานโปรแกรม

บทนี้อธิบายวิธีติดตั้ง รันโปรแกรม และวิธีเล่นเกม Tower Defense อย่างละเอียด

3.1 ความต้องการของระบบ (System Requirements)

รายการ	รายละเอียด
Java	Java Runtime Environment (JRE) 24 ขึ้นไป
JavaFX	JavaFX SDK 24 ขึ้นไป
ระบบปฏิบัติการ	Windows 10/11
RAM	อย่างน้อย 512 MB
พื้นที่ดิสก์	อย่างน้อย 50 MB

3.2 การติดตั้งและรันโปรแกรม

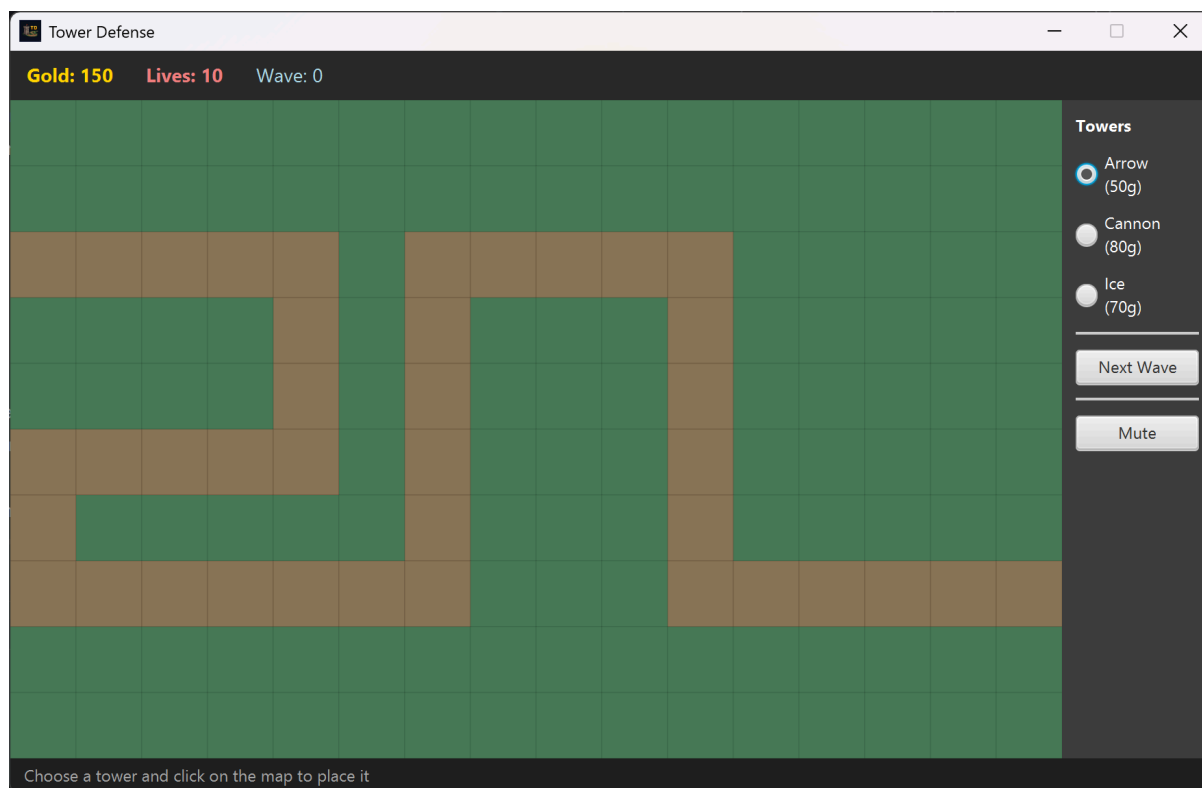
3.2.1 วิธีรันจาก JAR file

1. ดาวน์โหลดไฟล์ EKae-1-with-sources.jar จาก GitHub repository
2. สร้างโฟลเดอร์ใหม่ที่ไหนก็ได้ เช่น TowerDefense/
3. นำไฟล์ EKae-1-with-sources.jar วางไว้ในโฟลเดอร์ที่สร้าง
4. เปิด Command Prompt หรือ Terminal แล้ว cd เข้าโฟลเดอร์นั้น
5. พิมพ์คำสั่ง: `java --module-path <path-to-javafx-lib> --add-modules ALL-MODULE-PATH -jar EKae-1-with-sources.jar`
6. โปรแกรมจะเปิดขึ้นมาทันที

3.3 หน้าจอหลักของโปรแกรม

เมื่อรันโปรแกรมสำเร็จจะเห็นหน้าจอหลักแบ่งออกเป็น 3 ส่วนดังนี้

ส่วน	รายละเอียด
HUD (บน)	แสดง Gold, Lives และ Wave ปัจจุบัน
Map (กลาง)	พื้นที่เกม แสดงเส้นทางของ Enemy และ Tower ที่วางไว้
Shop (ขวา)	เลือกประเภท Tower, ปุ่ม Next Wave และปุ่ม Mute
Message Bar (ล่าง)	แสดงข้อความแจ้งเตือนต่าง ๆ เช่น วาง Tower สำเร็จหรือไม่



รูปที่ 3.1 หน้าจอหลักของโปรแกรม

3.4 HUD (Head-Up Display)

HUD แสดงข้อมูลสำคัญของเกมที่ด้านบนของหน้าจอ

องค์ประกอบ	สีที่แสดง	ความหมาย
Gold	สีทอง	จำนวนเงินที่มีอยู่ ใช้ซื้อ Tower
Lives	สีแดงอ่อน	จำนวนชีวิตที่เหลือ ลดลงเมื่อ Enemy ผ่านถึง base ถ้าหมดจะ Game Over
Wave	สีฟ้าอ่อน	หมายเลข Wave ปัจจุบัน

3.5 ประเภทของ Tower

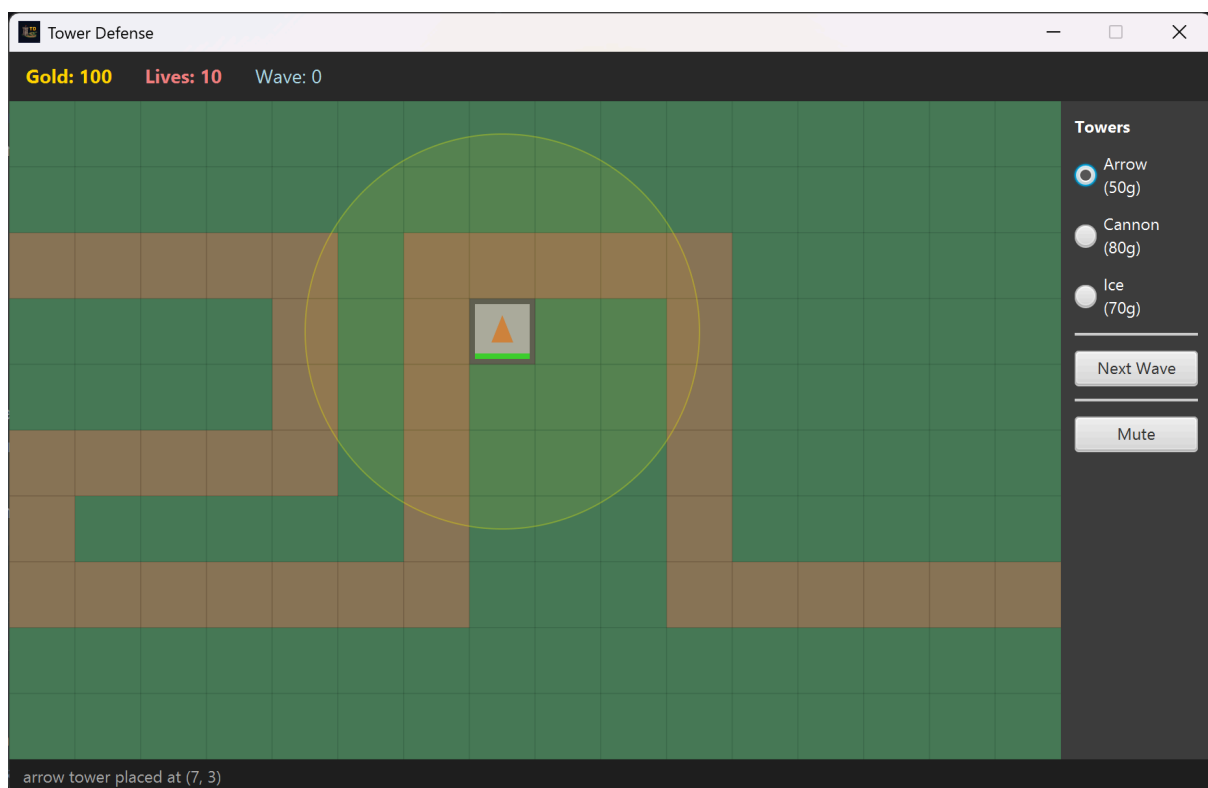
Tower แต่ละประเภทมีความสามารถและราคาที่แตกต่างกัน เลือกใช้ให้เหมาะกับสถานการณ์

Tower	ราคา	Damage	Range	ความสามารถพิเศษ
Arrow Tower	50 Gold	15	3 tiles	ยิง single target เหมาะวางจำนวนมากตั้งแต่ต้นเกม

Cannon Tower	80 Gold	20	2 tiles	ยิง splash โจมตีทุก Enemy ในรัศมีพร้อมกัน
Ice Tower	70 Gold	5	3 tiles	ทำให้ Enemy ช้าลง 60% นาน 2 วินาที

3.6 วิธีวาง Tower

- เลือกประเภท Tower จาก Shop panel ด้านขวา (Arrow / Cannon / Ice)
 - ตรวจสอบว่ามี Gold เพียงพอ
 - คลิกบน tile บน Map ที่ต้องการวาง
 - ถ้าวางสำเร็จ Tower จะปรากฏบน Map และ Gold จะถูกหัก
 - ถ้าวางไม่ได้ Message Bar จะแจ้งเหตุผล เช่น Gold ไม่พอ หรือ tile นั้นเป็นเส้นทาง Enemy
- หมายเหตุ: Tower จะแสดงวงกลมแสดงระยะโจมตีรอบตัวเองตลอดเวลา เพื่อช่วยในการวางแผน



รูปที่ 3.2 การเลือกและวาง Tower บน Map

3.7 ประเภทของ Enemy

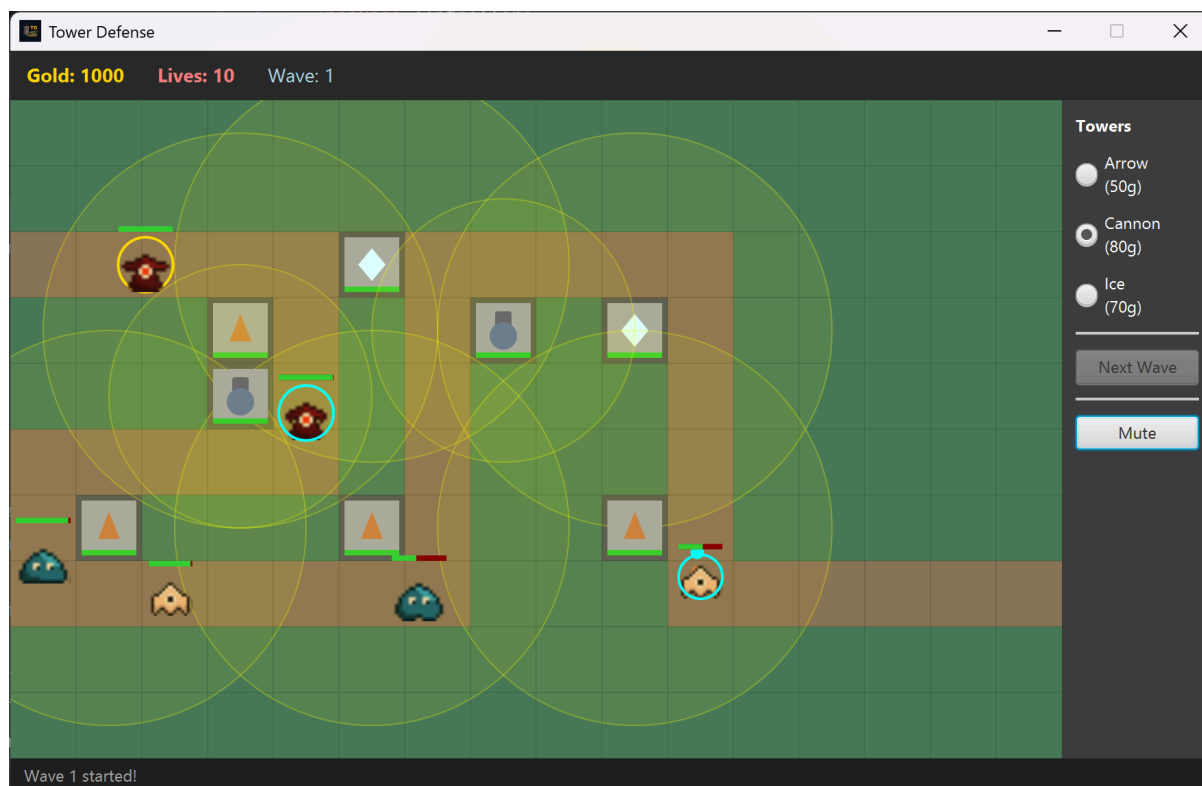
Enemy	HP	Speed	Reward	ความสามารถพิเศษ
Basic Enemy	100	ปกติ	10 Gold	Enemy พื้นฐาน ปรากฏในทุก Wave

Fast Enemy	50	เร็ว x2	15 Gold	วิ่งเร็วมาก HP น้อย ต้องวาง Tower ให้ครอบคลุม
Flying Enemy	80	เร็วกว่า Basic Enemy	20 Gold	บินได้ Cannon และ Ice Tower ยังไม่ถึง
Boss Enemy	500	ช้า	50 Gold	มี Armor ลด damage ที่รับ HP สูงมาก

3.8 ระบบ Wave

12. กดปุ่ม Next Wave ใน Shop panel เพื่อเริ่ม Wave ถัดไป
13. Enemy จะ spawn ที่ละตัวทุก 2 วินาที จนครบตามจำนวนของ Wave นั้น
14. ระหว่าง Wave กำลังดำเนินอยู่ปุ่ม Next Wave จะ disable ไม่สามารถกดได้
15. เมื่อ Enemy ทุกตัวใน Wave ตายหมดหรือผ่านไปแล้ว ปุ่ม Next Wave จะ enable กลับมา
16. Wave ที่สูงขึ้นจะมี Enemy จำนวนมากขึ้นและมี Fast Enemy และ Boss Enemy ปรากฏด้วย

Wave	Basic Enemy	Fast Enemy	Flying Enemy	Boss Enemy
Wave 1	7 ตัว	-	-	-
Wave 2	9 ตัว	4 ตัว	2 ตัว	-
Wave 3	11 ตัว	6 ตัว	6 ตัว	3 ตัว
Wave 4+	เพิ่มขึ้นเรื่อย ๆ	เพิ่มขึ้นเรื่อย ๆ	เพิ่มขึ้นเรื่อย ๆ	เพิ่มขึ้นเรื่อย ๆ



รูปที่ 3.3 ตัวอย่าง Wave ที่กำลังดำเนินอยู่

3.9 เงื่อนไขการชนะและแพ้

การแพ้ (Game Over)

เกมจะจบทันทีเมื่อ Lives ลดลงเหลือ 0 โดย Lives จะลดลง 1 ทุกครั้งที่ Enemy เดินผ่านถึง base ได้สำเร็จ หน้าจอจะแสดง GAME OVER และเสียง Game Over จะดัง



รูปที่ 3.4 หน้าจอ Game Over

กลยุทธ์แนะนำ

- วาง Arrow Tower จำนวนมากในช่วงต้นเพราะราคาถูก
- วาง Ice Tower ก่อน Cannon Tower เพื่อ slow Enemy ให้ Cannon ยิงได้นานขึ้น
- วาง Tower บนช่วง path ที่ Enemy ผ่านหลายครั้ง เช่น บริเวณที่เส้นทางหักมุม
- เก็บ Gold ไว้บ้างเพื่อซื้อ Tower รับมือ Enemy ใน Wave ถัดไป

3.10 การควบคุมเสียง

กดปุ่ม Mute ใน Shop panel เพื่อปิด/เปิดเสียงทั้งหมด รวมถึง BGM และ Sound Effect

เสียง	เกิดขึ้นเมื่อ
เสียงยิง (shoot)	Tower ยิงกระสุนออกไป
เสียง Game Over	Lives หมด เกมจบ
Background Media (BGM)	เล่น loop ตลอดเกม

บทที่ 4 การออกแบบเชิงวัตถุ (OOP)

บทนี้อธิบายหลักการเขียนโปรแกรมเชิงวัตถุที่นำมาใช้ในโครงงาน Tower Defense พร้อมเหตุผลในการออกแบบ

4.1 Encapsulation and Access Modifier

4.1.1 การใช้ Access Modifier

Access Modifier	ตัวอย่างที่ใช้	เหตุผล
private	field ทุกตัวใน subclass เช่น splashRadius, slowMultiplier, armor	ป้องกันการแก้ค่าโดยตรงจากภายนอก บังคับใช้ผ่าน getter/setter
protected	field ใน Entity และ Enemy เช่น hp, maxHp, pixelX, pixelY	ให้ subclass เข้าถึงได้โดยตรง เพราะ subclass ต้องใช้ในการ render และ move
public	method ที่เป็น API เช่น attack(), takeDamage(), render()	เปิดให้ class อื่น GameController และ Main เรียกใช้ได้
static final	constant ใน GameConfig เช่น TILE_SIZE, COLS, ROWS	เป็นค่าคงที่ที่ใช้ร่วมกันทั้งโครงงาน ไม่ควรแก้ไขได้

4.1.2 ตัวอย่างโค้ด

```
// Entity.java
public abstract class Entity {
    protected int tileX; // protected - subclass เข้าถึงได้
    protected int tileY; // protected - subclass เข้าถึงได้
    protected int hp; // protected - subclass ใช้ใน render
    private int maxHp; // private - แก้ผ่าน getMaxHp() เท่านั้น

    public int getHp() { return hp; } // public - เปิดให้ภายนอกอ่านได้
    public int getMaxHp() { return maxHp; } // public - เปิดให้ภายนอกอ่านได้
}

// BossEnemy.java
public class BossEnemy extends Entity {
    private int armor; // private - แก้ผ่าน setArmor() เท่านั้น
```

```

public void setArmor(int armor) { this.armor = armor; }
public int getArmor() { return armor; }
}

```

4.2 Inheritance

4.2.1 โครงสร้าง Inheritance

โครงงานนี้ออกแบบ class hierarchy แบบ 2 ชั้น โดยมี Entity เป็น base class ที่ Tower และ Enemy สืบทอดต่อ และแต่ละตัวยังมี subclass อีกชั้นหนึ่ง

สายที่ 1 - Tower	Entity -> Tower -> ArrowTower / CannonTower / IceTower
สายที่ 2 - Enemy	Entity -> Enemy -> BasicEnemy / FastEnemy / FlyingEnemy / BossEnemy

4.2.2 เหตุผลที่ใช้ Inheritance

- Entity เก็บ field ที่ Tower และ Enemy มีร่วมกันคือ tileX, tileY, hp และ maxHp ไว้ที่เดียว ไม่ต้องเขียนซ้ำในทุก subclass
- Tower เก็บ logic cooldown (isReady, resetCooldown) และ range ไว้ที่เดียว ArrowTower, CannonTower, IceTower ไม่ต้องเขียน cooldown เอง
- Enemy เก็บ movement (move), slow effect (applySlow, isSlowed) และ hp bar ไว้ที่เดียว subclass ทุกตัวได้ใช้โดยไม่ต้องเขียนซ้ำ

4.2.3 ตัวอย่างโค้ด

```

// Entity.java - base class
public abstract class Entity implements Renderable {
    protected int tileX;
    protected int tileY;
    protected int hp;
    protected int maxHp;
}

// Tower.java - สืบทอดจาก Entity
public abstract class Tower extends Entity implements Attackable {
    protected int range;
    protected int damage;
    public boolean isReady() { ... } // ทุก Tower ใช้ร่วมกัน
}

```



```
// ArrowTower.java - สืบทอดจาก Tower
public class ArrowTower extends Tower {
    public ArrowTower(TilePoint position) {
        super(position, 100, 3, 15, 50, 800);
    }
}
```

4.3 Polymorphism

4.3.1 ตัวอย่าง Polymorphism ที่ใช้ในโครงการ

Method เดียวกันในแต่ละ subclass ทำงานต่างกัน โดย GameController เรียก method เดิมโดยไม่ต้องรู้ว่าเป็น class ไດ

Method	Class	พฤติกรรม
attack(Enemy)	ArrowTower	ยิง single target สร้าง Projectile ธรรมดา
attack(Enemy)	CannonTower	ยิง splash สร้าง SplashProjectile โจมตีทุก Enemy ในรัศมี
attack(Enemy)	IceTower	ยิงแล้ว apply slow effect ให้ Enemy ที่โดน
move(double)	BasicEnemy	เดินด้วย speed ปกติ
move(double)	FastEnemy	เดินเร็วกว่า Basic 2 เท่า
move(double)	FlyingEnemy	เดินเร็วกว่า แต่โดนโจมตีได้เฉพาะ ArrowTower
move(double)	BossEnemy	เดินช้าแต่มี armor ลด damage ที่รับ
render(GraphicsContext)	แต่ละ Tower	วาด range circle ต่างกัน
render(GraphicsContext)	แต่ละ Enemy	วาด sprite และขนาดต่างกัน

4.3.2 ตัวอย่างโค้ด

```
// GameController - เรียก attack() โดยไม่ต้องรู้ว่าเป็น Tower ประเภทใด
for (Tower t : map.getTowers()) {
    List<Enemy> inRange = map.getEnemiesInRange(t);
    if (!inRange.isEmpty()) {
```

```

    Projectile p = t.attack(inRange.get(0)); // Polymorphism
    if (p != null) map.addProjectile(p);
  }
}

// ArrowTower, CannonTower, IceTower ต่างก็ override attack()
// แต่ GameController เรียกผ่าน Tower reference เดิมได้เลย

```

4.4 Abstraction และ Interface

4.4.1 Interface ที่ใช้

Interface	Method หลัก	ตัวอย่าง Class ที่ Implement
Attackable	attack(Enemy), getRange(), getDamage()	Tower ทุกประเภท
Damageable	takeDamage(int), isDead()	Enemy ทุกประเภท
Renderable	render(GraphicsContext)	Tower และ Enemy ทุกตัว

4.4.2 เหตุผลที่ใช้ Interface

- Renderable ทำให้ GameController สามารถเรียก render() บน Tower และ Enemy ได้โดยไม่ต้องรู้ว่าเป็น class ใด ลด coupling ระหว่าง class
- Attackable แยก contract การโจมตีออกจาก implementation ทำให้ในอนาคตสามารถเพิ่ม class ใหม่ที่โจมตีได้โดยไม่ต้องแก้ Tower
- Damageable ทำให้ Projectile สามารถเรียก takeDamage() บน target ได้โดยไม่ต้อง cast type

4.4.3 ตัวอย่างโค้ด

```

// Renderable.java
public interface Renderable {
    void render(GraphicsContext gc);
}

// GameController - เรียก render ได้ทั้ง Tower และ Enemy
// โดยไม่ต้องรู้ว่าเป็น class ใด
controller.getMap().getTowers().forEach(t -> t.render(gc));
controller.getMap().getEnemies().forEach(e -> e.render(gc));

```

4.5 Thread

4.5.1 Thread ที่ใช้ในโครงการ

Thread	Interval	หน้าที่
GameLoopThread	ทุก 16ms (~60fps)	เรียก controller.update() และ screen.render() สลับกัน
SpawnThread	ทุก 2 วินาที	เรียก waveController.spawnNext() เพื่อ spawn Enemy ทีละตัว

4.5.2 เหตุผลที่ใช้ 2 Thread แยกกัน

- ถ้า spawn Enemy ใน GameLoopThread โดยตรง การ sleep 2 วินาทีจะทำให้เกมค้าง 2 วินาที ทุกครั้งที่ spawn
- แยก SpawnThread ออกมาทำให้ game loop วิ่งได้ต่อเนื่อง 60fps ตลอดเวลาโดยไม่ถูก block
- ใช้ Collections.synchronizedList() สำหรับ List ของ Tower และ Enemy เพื่อป้องกัน race condition เมื่อทั้ง 2 Thread เข้าถึงพร้อมกัน
- ใช้ volatile boolean สำหรับ running และ paused flag เพื่อให้ Thread เห็นค่าล่าสุดเสมอ
- เรียก Platform.runLater() ทุกครั้งที่วาดบน Canvas เพราะ JavaFX กำหนดให้การแก้ไข UI ต้องทำบน JavaFX Application Thread เท่านั้น

4.5.3 ตัวอย่างโค้ด

```
// GameLoopThread.java
public class GameLoopThread extends Thread {
    private static final long TARGET_FPS_MS = 16;
    private volatile boolean running = true; // volatile - thread อื่นเห็นค่าล่าสุด
    private volatile boolean paused = false;

    public void run() {
        while (running) {
            if (paused) { Thread.sleep(100); continue; }
            long start = System.currentTimeMillis();
            controller.update(); // update logic
            screen.render(); // render ผ่าน Platform.runLater()
            long sleep = TARGET_FPS_MS - (System.currentTimeMillis() - start);
            if (sleep > 0) Thread.sleep(sleep);
        }
    }
}
```

```
}  
  
// Main.java - บังคับวาดบน JavaFX Application Thread  
public void render() {  
    Platform.runLater() -> {  
        gc.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());  
        drawMap();  
        drawTowers();  
        drawEnemies();  
    };  
}
```

บทที่ 5 รายละเอียด Class ในโครงการ

บทนี้อธิบาย class ทุกตัวในโปรเจกต์ แบ่งตาม package พร้อม field, constructor และ method ที่สำคัญ

5.1 ภาพรวม Package และ Class

Class	Package	หน้าที่
Main	main	Entry point จัดการ UI ทั้งหมด เริ่ม Thread
GameController	controller	Logic หลักของเกม update enemy tower gold lives
WaveController	controller	จัดการ wave และ spawn enemy
Entity	model.entity	Abstract base class ของ Tower และ Enemy
Tower	model.entity.tower	Abstract class ของ Tower จัดการ cooldown range
ArrowTower	model.entity.tower	Tower ยิง single target
CannonTower	model.entity.tower	Tower ยิง splash damage
IceTower	model.entity.tower	Tower ทำให้ enemy ช้ำ
Enemy	model.entity.enemy	Abstract class ของ Enemy จัดการ movement slow
BasicEnemy	model.entity.enemy	Enemy พื้นฐาน speed และ HP ปกติ
FastEnemy	model.entity.enemy	Enemy วิ่งเร็ว HP น้อย
FlyingEnemy	model.entity.enemy	Enemy บินได้ Cannon และ Ice Tower ยิงไม่ถึง, มี shadow render และ hover offset
BossEnemy	model.entity.enemy	Enemy HP สูง มี armor
GameMap	model	เก็บ state ของ map grid path tower enemy
Projectile	model	กระสุนที่ tower ยิงออกไป

SplashProjectile	model	กระสุน splash ของ CannonTower
TilePoint	model	แทน coordinate ของ tile บน map
TowerFactory	model.factory	Factory สร้าง Tower ตาม type
GameLoopThread	thread	Thread หลัก update + render 60fps
SpawnThread	thread	Thread spawn enemy ทุก 2 วินาที
Attackable	interfaces	Interface สำหรับ object ที่โจมตีได้
Damageable	interfaces	Interface สำหรับ object ที่รับ damage ได้
Renderable	interfaces	Interface สำหรับ object ที่วาดตัวเองได้
GameConfig	util	Constant ที่ใช้ร่วมกันทั้งโปรเจกต์
SoundManager	util	จัดการเสียงทั้งหมด Singleton pattern
SpriteSheet	util	จัดการ frame animation จาก sprite sheet

5.2 Interfaces

Attackable

Interface สำหรับ object ที่สามารถโจมตีได้ Tower ทุกประเภท implement

Method	คำอธิบาย
attack(target: Enemy): Projectile	โจมตี enemy และ return Projectile หรือ null ถ้า cooldown ยังไม่หมด
getRange(): int	ค่านั้ระยะโจมตีเป็นจำนวน tile
getDamage(): int	ค่านั้ damage ที่จะ deal ต่อครั้ง

Damageable

Interface สำหรับ object ที่รับ damage ได้ Enemy ทุกประเภท implement

Method	คำอธิบาย
takeDamage(dmg: int): void	ลด HP ตาม damage ที่ได้รับ ถ้า HP ≤ 0 ให้ set isDead เป็น true

isDead(): boolean	คืนค่า true ถ้า HP เหลือ 0
-------------------	----------------------------

Renderable

Interface สำหรับ object ที่วาดตัวเองบน Canvas ได้ Tower และ Enemy ทุกตัว implement

Method	คำอธิบาย
render(gc: GraphicsContext): void	วาด object ลงบน canvas ณ ตำแหน่งปัจจุบัน

5.3 Model Classes

Entity (abstract)

Base class ของทุก object ในเกม เก็บ position และ HP

Field	Type	คำอธิบาย
tileX, tileY(protected)	int	ตำแหน่ง tile ของ entity
hp (protected)	int	HP ปัจจุบัน
maxHp (protected)	int	HP สูงสุด ใช้คำนวณ HP bar

Tower (abstract)

Abstract class ของ Tower ทุกประเภท จัดการ cooldown และ range การ render base tower และ HP bar

Field / Method	Type	คำอธิบาย
range (protected)	int	ระยะโจมตีเป็น tile
damage (protected)	int	damage ต่อครั้ง
cost (protected)	int	ราคาซื้อ Tower
fireRateMs (protected)	long	cooldown ระหว่างการยิง หน่วย millisecond
canHitFlying (private)	boolean	ยิง FlyingEnemy ได้หรือไม่
isReady()	boolean	คืน true ถ้าพ้น cooldown แล้ว
resetCooldown()	void	บันทึกเวลายิงล่าสุด + เล่นเสียงยิง
renderRange(gc)	void	วาด range circle รอบ Tower

attack(target)*	Projectile	abstract - subclass implement เพื่อสร้าง Projectile
-----------------	------------	---

ArrowTower / CannonTower / IceTower

Class	Cost	Damage	Range	ความพิเศษ
ArrowTower	50g	15	3	ยิง single target สร้าง Projectile ธรรมดา
CannonTower	80g	20	2	ยิง splash สร้าง SplashProjectile โจมตีทุก enemy ในรัศมี 60px
IceTower	70g	5	3	ยิงแล้ว applySlow(0.4, 2000) ให้ enemy ที่โดน

Enemy (abstract)

Abstract class ของ Enemy ทุกประเภท จัดการ smooth movement ตาม path, slow effect และ sprite animation

Field / Method	Type	คำอธิบาย
speed (protected)	double	ความเร็วเป็น tile ต่อวินาที
reward (protected)	int	Gold ที่ได้เมื่อ enemy ตาย
pixelX, pixelY (protected)	double	ตำแหน่ง pixel จริงที่ใช้วาด ทำให้ movement smooth
path (protected)	List<TilePoint>	เส้นทางที่ enemy เดินตาม
move(deltaTime)	void	เลื่อน pixelX/Y ไปหา tile ถัดไปที่ละ frame
takeDamage(dmg)	void	ลด HP + เล่นเสียงถ้าตาย
applySlow(mult, ms)	void	ลด speed ชั่วคราว
isSlowed()	boolean	คืน true ถ้า slow effect ยังมีผลอยู่
getBodyColor()*	Color	abstract - subclass กำหนดสี fallback

BasicEnemy / FastEnemy / BossEnemy

Class	HP	Speed	Reward	ความพิเศษ
-------	----	-------	--------	-----------

BasicEnemy	100	2.5	10g	Enemy พื้นฐาน ปรากฏในทุก Wave
FastEnemy	50	5	15g	วิ่งเร็วกว่า Basic 2 เท่า frame delay ลดเหลือ 80ms
FlyingEnemy	80	3.5	20g	มีเพียง ArrowTower ที่โจมตีมันได้
BossEnemy	500	1.5	50g	มี armor = 10 ลด damage ที่รับ minimum 1 เสมอ

GameMap

เก็บ state ของ map ทั้งหมด ใช้ synchronizedList เพื่อรองรับการเข้าถึงจากหลาย Thread

Method	คำอธิบาย
canPlace(col, row)	คืน true ถ้า tile เป็น grass และไม่มี Tower อยู่แล้ว
addTower(t)	เพิ่ม Tower และ set grid เป็น 2
addEnemy(e)	เพิ่ม Enemy เข้า synchronized list
getEnemiesInRange(t)	คืน list ของ Enemy ที่อยู่ใน range pixel ของ Tower
removeDead()	ลบ Enemy ที่ isDead() == true ออก
removeHitProjectiles()	ลบ Projectile ที่ isHit() == true ออก

Projectile และ SplashProjectile

Field / Method	Type	คำอธิบาย
update(deltaTime)	void	เลื่อนกระสุนไปหา target ถ้าถึงแล้ว deal damage + set isHit
render(gc)	void	วาดกระสุนบน canvas ถ้า isHit == false
isHit()	boolean	คืน true ถ้ากระสุนถึง target แล้ว
SplashProjectile.update()	void	override - ตอน hit deal damage กับทุก enemy ในรัศมี splashRadius

5.4 Controllers

GameController

จัดการ logic หลักของเกมทั้งหมด GameLoopThread เรียก update() ทุก 16ms

Method	คำอธิบาย
update()	วน move enemy, tower attack, update projectile, เก็บ gold, เช็ค lives, ลบ dead
buyTower(type, col, row)	เช็ค gold และ canPlace แล้ว add Tower ผ่าน TowerFactory
isGameOver()	คืน true ถ้า lives ≤ 0
resetUpdateTime()	reset lastUpdateTime หลัง resume เพื่อป้องกัน deltaTime กระโดด

WaveController

จัดการ wave ของเกม ควบคุมการ spawn enemy และลือกปุ่ม Next Wave ระหว่าง wave ดำเนินอยู่

Method	คำอธิบาย
nextWave()	เพิ่ม wave counter และ build spawn queue ถ้า wave ยังไม่ progress
spawnNext()	pop enemy จาก queue แล้ว add เข้า GameMap
isWaveComplete()	คืน true ถ้า queue ว่างและ map ไม่มี enemy เหลือ

5.5 Threads

GameLoopThread

Method	คำอธิบาย
run()	loop: update -> render -> sleep 16ms ถ้า paused ให้ sleep 100ms แทน
stopLoop()	set running = false หยุด thread
pauseLoop()	set paused = true ใช้ตอน window ไม่ได้ focus
resumeLoop()	set paused = false + resetUpdateTime() ป้องกัน deltaTime

	กระโดด
--	--------

SpawnThread

Method	คำอธิบาย
run()	loop: sleep intervalMs (default 2000ms) -> waveController.spawnNext()
stopSpawn()	set running = false หยุด thread
setIntervalMs(ms)	แก้ interval การ spawn ค่า default 2000ms

5.6 Utility Classes

SoundManager

จัดการเสียงทั้งหมดในเกม ใช้ Singleton pattern ทุก class เรียกผ่าน

SoundManager.getInstance()

Method	คำอธิบาย
play(key)	เล่น sfx โดย rewind ก่อนเสมอ shoot ใช้ pool 5 ตัวเพื่อรองรับพร้อมกัน
playBgm() / stopBgm()	เล่น / หยุด background music loop
switchToBattle()	หยุด bgm แล้วเล่น game_running ตอน wave เริ่ม
switchToMenu()	หยุด game_running แล้วเล่น bgm ตอน wave จบ
setMuted(bool)	toggle เสียงทั้งหมด

SpriteSheet

Utility class สำหรับตัดและแสดง frame จาก sprite sheet แนวนอน ใช้กับ Enemy ทุกประเภท

Method	คำอธิบาย
draw(gc, x, y, size)	ตัด frame ปัจจุบันจาก sheet แล้ววาดที่ตำแหน่งที่กำหนด
setFrameDelayMs(ms)	แก้ความเร็ว animation default 150ms FastEnemy ใช้ 80ms

บทที่ 6 การทดสอบโปรแกรม

โปรเจกต์นี้เขียน Unit Test ด้วย JUnit 5 ครอบคลุม class ที่มี logic หลักทั้งหมด แบ่งออกเป็น 3 package

6.1 Class ที่ทดสอบ

Test Class	Target Class	สิ่งที่ทดสอบ
ArrowTowerTest	ArrowTower	attack return Projectile, cooldown, damage, range, cost
CannonTowerTest	CannonTower	splash damage, enemiesRef, SplashProjectile type, ยิง FlyingEnemy ไม่ได้
IceTowerTest	IceTower	slow effect หลัง hit, damage ต่ำ, cooldown, ยิง FlyingEnemy ไม่ได้
TowerFactoryTest	TowerFactory	create ถูก class, position, case insensitive, unknown type
BasicEnemyTest	BasicEnemy	hp, takeDamage, isDead, move, applySlow
FastEnemyTest	FastEnemy	speed สูงกว่า Basic, slow effect, slow expiry
FlyingEnemyTest	FlyingEnemy	hp น้อยกว่า Basic, reward มากกว่า, move, slow
BossEnemyTest	BossEnemy	armor ลด damage, minimum 1, hp สูง, reward สูง
ProjectileTest	Projectile	เคลื่อนที่, hit target, deal damage, ไม่ damage ซ้ำ
GameMapTest	GameMap	canPlace, addTower, removeDead, getEnemiesInRange
WaveControllerTest	WaveController	nextWave, spawnNext, isWaveComplete, lock ระหว่าง wave
GameControllerTest	GameController	buyTower, gold หัก, lives, isGameOver, update

6.2 ผลการทดสอบ

ดูผลการทดสอบและรายละเอียด test case ทั้งหมดได้ที่ link ด้านล่าง

รายการ	Link	คำอธิบาย
Source Code Test	Link to Source Code Test	โค้ด JUnit ทุก class

ผลการ Run Test	Link to Result	ผล JUnit
----------------	--------------------------------	----------

บทที่ 7 ข้อจำกัดและแนวทางพัฒนาต่อ

บทนี้สรุปข้อจำกัดที่พบระหว่างการพัฒนาและ feature ที่สามารถเพิ่มเติมได้ในอนาคต เพื่อให้โปรแกรมมีความสมบูรณ์และประสบการณ์การเล่นที่ดีขึ้น

7.1 สรุปภาพรวม

หัวข้อ	สถานะปัจจุบัน	แนวทางพัฒนา
Reset เกม	ต้องปิดและเปิดโปรแกรมใหม่เพื่อเริ่มเกมใหม่	เพิ่มปุ่ม Reset ที่ล่าง GameMap, WaveController และ GameController แล้ว restart thread
ถอน Tower	วาง Tower แล้วไม่สามารถเอาออกได้	เพิ่ม mode ถอน Tower โดย refund gold บางส่วน เช่น 50% ของราคา
Projectile หาย	Projectile หายไปทันทีเมื่อ target ตายก่อนถึง	ให้ Projectile เดินหน้าต่อและ hit enemy ตัวถัดไปที่อยู่ใกล้สุด หรือวิ่งต่อจนสุดระยะแล้วค่อยหาย
Cannon splash	CannonTower ยิง Projectile ที่วิ่งตรงไป target แล้ว splash ทันที	ทำให้ Projectile บินเป็นโค้ง (arc) แล้ว splash เมื่อตกถึงพื้น จะดูสมจริงกว่า
Fast Forward	เกมวิ่งที่ความเร็วคงที่ตลอด	เพิ่มปุ่ม 2x หรือ 3x โดยคูณ deltaTime และ intervalMs ของ SpawnThread ตามสัดส่วน
Game Balance	ค่า damage, hp, wave scaling ยังไม่ได้ balance อย่างจริงจัง	ปรับ formula ของ WaveController ให้ difficulty curve สมเหตุสมผล และ playtesting

7.2 รายละเอียดแต่ละหัวข้อ

7.2.1 ปุ่ม Reset เกม

ปัจจุบันถ้าต้องการเริ่มเกมใหม่ผู้เล่นต้องปิดแล้วเปิดโปรแกรมใหม่ทั้งหมด ซึ่งไม่สะดวกโดยเฉพาะหลัง Game Over แนวทางแก้ไขคือเพิ่มปุ่ม Restart ใน Shop panel โดยสร้าง GameController และ GameMap ใหม่ แล้ว assign ให้ GameLoopThread และ SpawnThread ที่กำลังวิ่งอยู่

- ความยาก: ปานกลาง ต้องระวังเรื่อง thread safety ตอน swap controller กลางคัน
- OOP ที่เกี่ยวข้อง: GameController และ GameMap ต้องออกแบบให้ reset ได้โดยไม่ต้อง restart thread

7.2.2 ปุ่มถอน Tower

เมื่อผู้เล่นวาง Tower ผิดตำแหน่งหรือต้องการเปลี่ยน Tower ไม่มีทางแก้ไขได้เลย แนวทางคือเพิ่ม mode ถอน Tower โดยกดปุ่ม Sell แล้วคลิก Tower ที่ต้องการ โปรแกรมจะลบ Tower ออกจาก GameMap และคืน gold บางส่วนให้ผู้เล่นหรือไม่คืนเลยก็ได้

- ความยาก: ง่าย เพิ่ม method `removeTower(col, row)` ใน GameMap และอัปเดต grid กลับเป็น GRASS
- OOP ที่เกี่ยวข้อง: GameMap และ GameController เพิ่ม method ใหม่โดยไม่กระทบ class อื่น

7.2.3 พฤติกรรม Projectile เมื่อ Target ตาย

ปัจจุบันถ้า Enemy ตายก่อนที่ Projectile จะถึง Projectile จะหายไปทันที ซึ่งดูไม่เป็นธรรมชาติ โดยเฉพาะ CannonTower ที่ยิง Projectile เข้า และมีโอกาสที่ target จะตายจาก Tower อื่นก่อน ทำให้ splash damage ไม่เกิดขึ้น

- แนวทางที่ 1: ให้ Projectile วิ่งต่อแล้ว retarget Enemy ตัวที่อยู่ใกล้สุด เหมาะกับ ArrowTower
- แนวทางที่ 2: ให้ Projectile วิ่งไปถึงตำแหน่งที่ target อยู่ตอนตาย แล้ว splash ณ จุดนั้น เหมาะกับ CannonTower
- แนวทางที่ 3: ให้ CannonTower ยิง Projectile แบบ arc โค้งไปตกที่ตำแหน่ง tile เป้าหมาย ไม่ track enemy เลย

7.2.4 Fast Forward

เกมในปัจจุบันวิ่งที่ความเร็วคงที่ตลอด ผู้เล่นที่วาง Tower ครบแล้วต้องรอ Enemy เดินถึง base ซึ่งอาจรู้สึกช้า โดยเฉพาะ wave ต้น ๆ แนวทางคือเพิ่มปุ่ม 2x โดยคูณ deltaTime ที่ส่งให้ `enemy.move()` และลด intervalMs ของ SpawnThread ตามสัดส่วน

- ความยาก: ปานกลาง ต้องระวังว่า `Thread.sleep()` ของ SpawnThread ต้องปรับตาม speed multiplier ด้วย
- OOP ที่เกี่ยวข้อง: เพิ่ม speedMultiplier ใน GameController แล้วส่งต่อไปยัง `SpawnThread.setIntervalMs()`

7.2.5 Game Balance

ค่าความยากของเกมในปัจจุบันใช้ formula ง่าย ๆ ใน WaveController ซึ่งยังไม่ได้ balance อย่างจริงจัง ผู้เล่นอาจรู้สึกว่าเกมง่ายหรือยากเกินไปในบาง wave สิ่งที่ต้องปรับได้แก่

- Tower damage vs Enemy HP: ปรับให้ผู้เล่นต้องใช้ Tower หลายตัวจริง ๆ ไม่ใช่แค่ Arrow Tower เยอะ ๆ
- Wave scaling: ปรับ formula ให้ difficulty เพิ่มขึ้นแบบ smooth ไม่กระโดดชันเกินไป
- Gold economy: ปรับ reward จาก enemy และราคา tower ให้ผู้เล่นต้องตัดสินใจ trade-off จริง ๆ
- Boss frequency: Boss ปัจจุบันเริ่ม wave 3 อาจเร็วเกินไป ควรเลื่อนเป็น wave 5 หรือมากกว่า

7.3 ข้อจำกัดที่รับรู้

นอกจากสิ่งที่พัฒนาต่อได้แล้ว ยังมีข้อจำกัดจาก platform ที่ไม่สามารถแก้ไขได้ง่ายในขอบเขตโปรเจกต์นี้

- หน้าจอค้างขณะกดปุ่ม minimize ค้าง: เป็น OS-level behavior ของ Windows ที่ block JavaFX Application Thread ระหว่าง modal loop ของ window decoration game logic ยังทำงานถูกต้องตลอด แค่ render หยุดชั่วคราว แก้สมบรณ์ได้ต้องใช้ WritableImage + render บน background thread
- AudioClip ไม่รองรับ .mp3 บาง platform: อาจทำให้ไม่มีเสียง sfx ได้ในบาง platform ซึ่งทำให้ประสบการณ์การเล่นไม่สมบูรณ์เท่าที่ควร แนวทางหนึ่งที่แก้ไขได้คือการใช้ไฟล์ sfx ที่เป็นนามสกุล .wav
- Path แบบ hardcode: เส้นทางของ enemy เขียน hardcode ใน GameMap ไม่รองรับ map หลากรูปแบบ ถ้าต้องการหลาย map ต้องออกแบบระบบโหลด path จากไฟล์

ภาคผนวก

ก. JavaDoc

เอกสาร JavaDoc ที่ generate จาก source code สามารถเปิดดูได้ที่ link ด้านล่าง

รายการ	Link	คำอธิบาย
JavaDoc	Link to Javadoc	ไฟล์หลักคือ index.html

ข. GitHub Repository

Source code, JAR file, Report และ UML Diagram ทั้งหมดอยู่ใน repository นี้

รายการ	Link	คำอธิบาย
GitHub Repository	Link to GitHub Repository	ไฟล์หลักคือ index.html
JAR file	Link to JAR file	ดาวน์โหลดได้จาก build/libs
UML Diagram	Link to UML	ไฟล์ .puml และ .png
Report	Link to Report	ไฟล์รายงาน

ค. แหล่งที่มาของ Asset

ประเภท	แหล่งที่มา
Sprite Images	Link to Sprite Images
Sound Effects	Pixabay
Background Music	Pixabay